

EXPERIMENT NO.6

Name: Niraj Kothawade

Class:D20A

Roll No:30

Batch:B

Aim:Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** The contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Implementation:

The project “**Land Ownership Registry System**” focuses on automating land records and ownership transfers using Ethereum-based Smart Contracts.

To ensure the system works, we will implement two primary contracts:

1. **LandRegistry.sol:** Manages the database of lands and their owners.
2. **LandTransfer.sol:** Handles the logic of transferring ownership from a Seller to a Buyer.

Code:-

1. LandRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LandRegistry {
    struct Land {
        uint id;
        string location;
        uint area;
        address owner;
        bool isRegistered;
    }

    mapping(uint => Land) public lands;
    address public admin;

    constructor() {
        admin = msg.sender;
    }

    function registerLand(uint _id, string memory _location, uint _area, address _owner)
    public {
        require(msg.sender == admin, "Only admin can register land");
        require(!lands[_id].isRegistered, "Land already registered");

        lands[_id] = Land(_id, _location, _area, _owner, true);
    }

    function updateOwner(uint _id, address _newOwner) public {
        // This function will be called by the Transfer contract
        lands[_id].owner = _newOwner;
    }

    function getLand(uint _id) public view returns (uint, string memory, uint, address) {
        Land memory l = lands[_id];
```

```
        return (l.id, l.location, l.area, l.owner);
    }
}
```

2.LandTransfer.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
import "./LandRegistry.sol";
```

```
contract LandTransfer {
    LandRegistry public registry;
```

```
    constructor(address _registryAddress) {
        registry = LandRegistry(_registryAddress);
    }
```

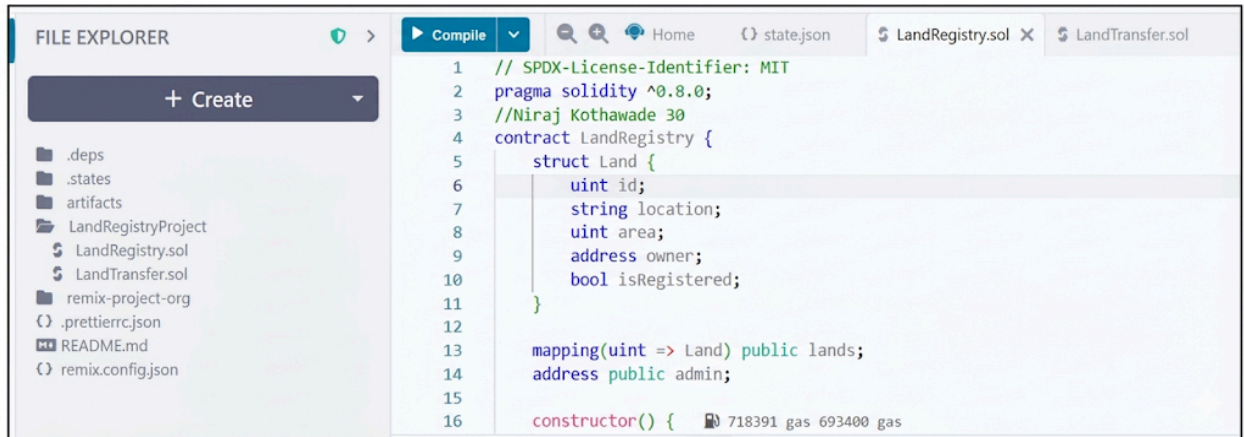
```
    function transferOwnership(uint _landId, address _newOwner) public {
        (,, address currentOwner) = registry.getLand(_landId);
```

```
        require(msg.sender == currentOwner, "Only the owner can sell this land");
```

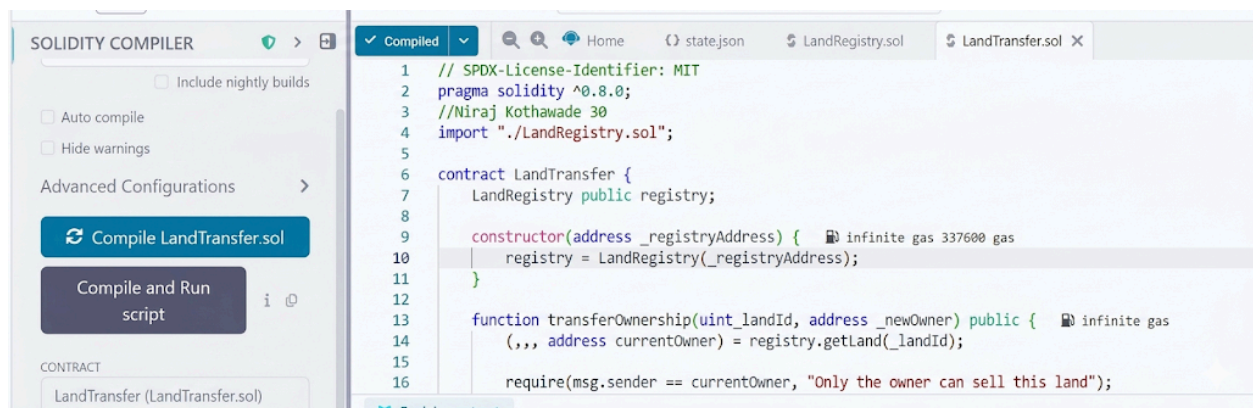
```
        registry.updateOwner(_landId, _newOwner);
    }
}
```

Step 1: Set up the Files

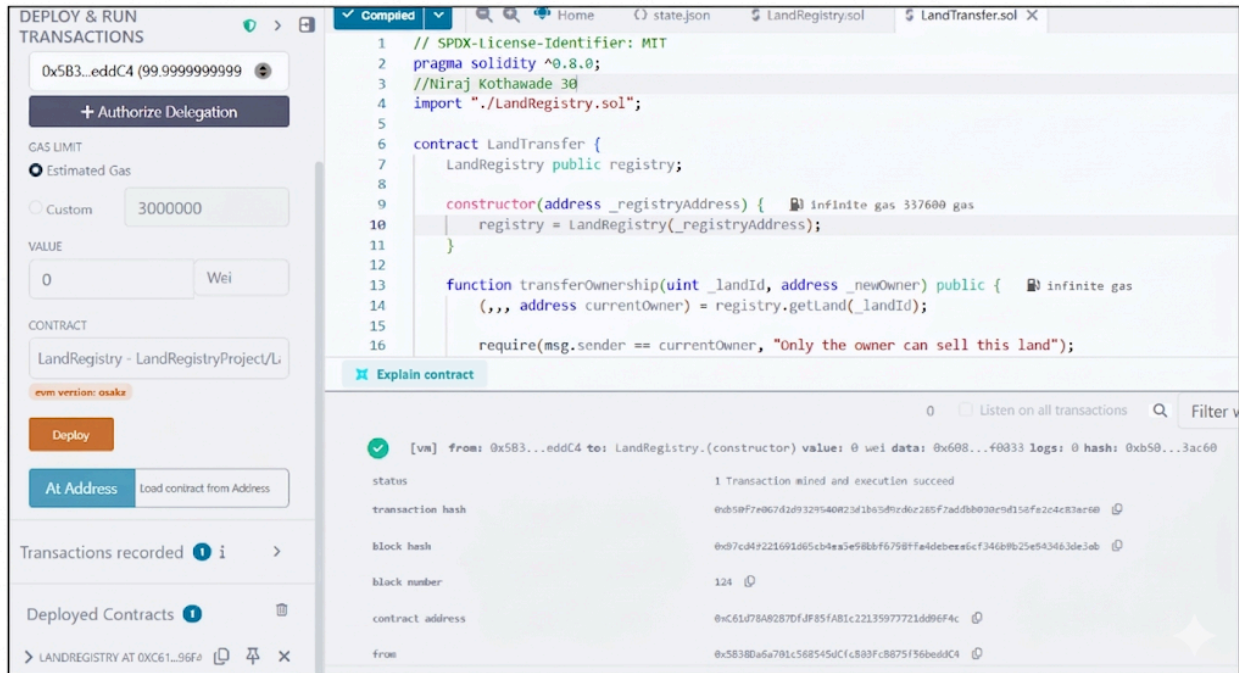
1. Go to Remix IDE.
2. In the File Explorer, create a new folder named LandRegistryProject.
3. Create two files: LandRegistry.sol and LandTransfer.sol.



Step 2: Compile both the contracts



Step 3: Deploy the LandRegistry contract



DEPLOY & RUN TRANSACTIONS

0x5B3...eddC4 (99.9999999999)

+ Authorize Delegation

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

CONTRACT

LandRegistry - LandRegistryProject/L

evm version: osaka

Deploy

At Address Load contract from Address

Transactions recorded 1 i >

Deployed Contracts

> LANDREGISTRY AT 0XC61...96F4

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3 //Niraj Kothawade 30
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) { infinite gas 337600 gas
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address _newOwner) public { infinite gas
14         (,,, address currentOwner) = registry.getLand(_landId);
15
16         require(msg.sender == currentOwner, "Only the owner can sell this land");
```

Explain contract

0 ☐ Listen on all transactions

[vm] from: 0x5B3...eddC4 to: LandRegistry.(constructor) value: 0 wei data: 0x608...f0833 logs: 0 hash: 0xb50...3ac60

status 1 Transaction mined and execution succeed

transaction hash 0xb50f7e067d2d9329540823d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60

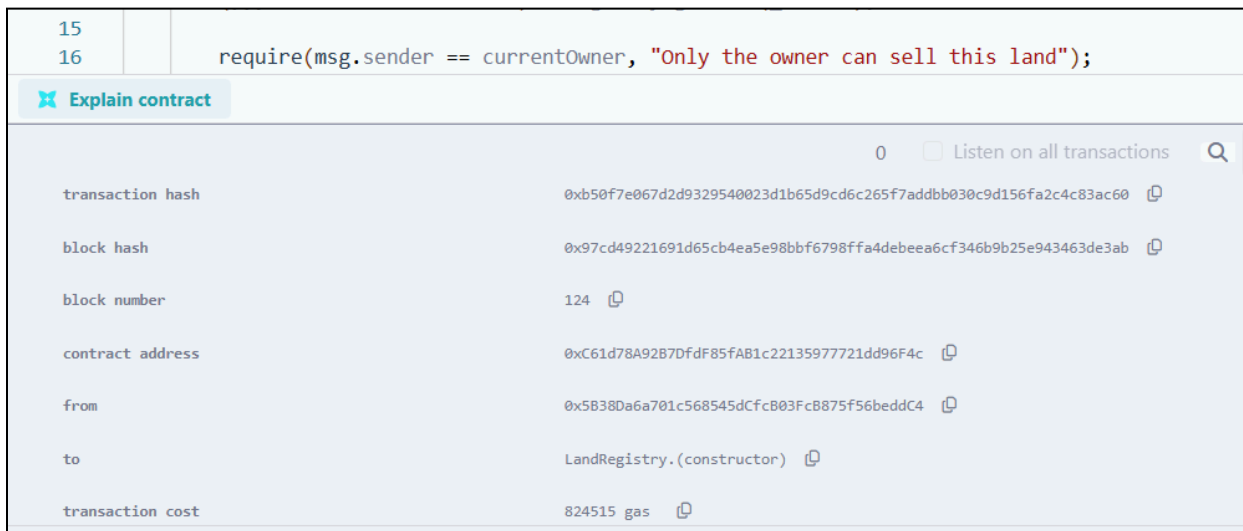
block hash 0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab

block number 124

contract address 0xC61d78A92B7Df85fAB1c2213597721dd96F4c

from 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4

Step 4: Copy the contract address of the deployed LandRegistry contract



```
15
16 require(msg.sender == currentOwner, "Only the owner can sell this land");
```

Explain contract

0 ☐ Listen on all transactions

transaction hash 0xb50f7e067d2d9329540823d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60

block hash 0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab

block number 124

contract address 0xC61d78A92B7Df85fAB1c2213597721dd96F4c

from 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4

to LandRegistry.(constructor)

transaction cost 824515 gas

Step 5: Select LandTransfer here in the dropdown

The screenshot displays the Remix IDE interface during the deployment of a Solidity contract. On the left sidebar, the 'CONTRACT' section shows a dropdown menu with three options: 'LandTransfer - LandRegistryProject/Li...', 'LandRegistry - LandRegistryProject/LandRegistry.sol', and 'LandTransfer - LandRegistryProject/LandTransfer.sol'. The third option is selected and highlighted in blue. Below the dropdown, there are buttons for 'At Address' and 'Load contract from Address'. The 'GAS LIMIT' section is set to 'Estimated Gas' with a value of 3000000. The 'VALUE' section is set to 0 Wei. The main editor shows the Solidity code for the 'LandTransfer' contract, which includes an import statement, a constructor, and a 'transferOwnership' function. The bottom panel shows the 'Transactions recorded' section with two entries: 'LANDREGISTRY AT 0XC61...96F4' and 'LANDTRANSFER AT 0XA2E...DCf'. The 'Deployed Contracts' section shows the 'LandTransfer' contract deployed at address 0XA2E...DCf. The right panel shows the transaction details for the deployment, including the status, transaction hash, block hash, block number, and contract address.

```
4 import "../LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) {
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address _newOwner) public {
14         if (msg.sender != registry.getLand(_landId).owner) {
15             revert("Only the owner can sell this land");
16         }
17         registry.transferOwnership(_landId, _newOwner);
18     }
19 }
```

creation of LandRegistry pending...

✓ [vm] from: 0x583...eddC4 to: LandRegistry.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xb50...3ac60

status 1 Transaction mined and execution succeed

transaction hash 0xb50f7e067d2d9329540023d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60

block hash 0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab

block number 124

contract address 0xC61d78A9287DfdF85FAB1c22135977721dd96F4c

Step 6: Add the contract address in place of _registryAddress field next to the deploy button, and click on deploy so as to deploy the LandTransfer contract

DEPLOY & RUN
TRANSACTIONS

+ Authorize Delegation

GAS LIMIT

☒ Estimated Gas

☐ Custom

3000000

VALUE

0

Wei

CONTRACT

LandTransfer - LandRegistryProject/Li

evm version: osaka

Deploy

0xC61d78A92B7DfdF85fAB1c22

▼

At Address

Load contract from Address

✓ [vm] from: 0x5B3...eddC4 to: LandTransfer.(constructor) value: 0 wei data: 0x608...96f4c logs: 0 hash: 0x02b...545c1

Debug

status	1 Transaction mined and execution succeed
transaction hash	0x02bfc4bb38722b91c6980f77829c9fe52aa29f2f6e96765034aa9b1a3f4545c1
block hash	0x30d868035e979b6362cde68e0892bca6d6a6f80774dc8473ada8cb5da89296bb
block number	125
contract address	0xa2e9669fc58d0550aF1BaEd20dcF10A9e00Cb97
from	0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4
to	LandTransfer.(constructor)
transaction cost	443826 gas
execution cost	360488 gas
output	0x608060d0c73d801561000f575f5f45b50600d361061003d575f3560a01c806370507f731d6100385780637b1030001d61005d575b5f5ff45b610057600d803603810

Step 7: Execute the transactions

1.Land Registration

In the deployed contracts section expand the LandRegistry contract, and add details as mentioned. Also ensure that the wallet address is added in the owner field. Once all this is done, click on transact to complete the transaction.

Deployed Contracts 2

LANDREGISTRY AT 0XC61...96F

Balance: 0 ETH

REGISTERLAND

_id: 1

_location: "Mumbai"

_area: 500

_owner: 0x5B38Da6a701c568545dCfcB03Fcd

Calldata

Parameters

transact

 Reset Sta



+   



0x5B3...eddC4 (99.999999999999746)

0xAb8...35cb2 (100.0 ETH)

0x4B2...C02db (100.0 ETH)

0x787...cabaB (100.0 ETH)

0x617...5E7f2 (100.0 ETH)

0x17F...8c372 (100.0 ETH)

0x5c6...21678 (100.0 ETH)

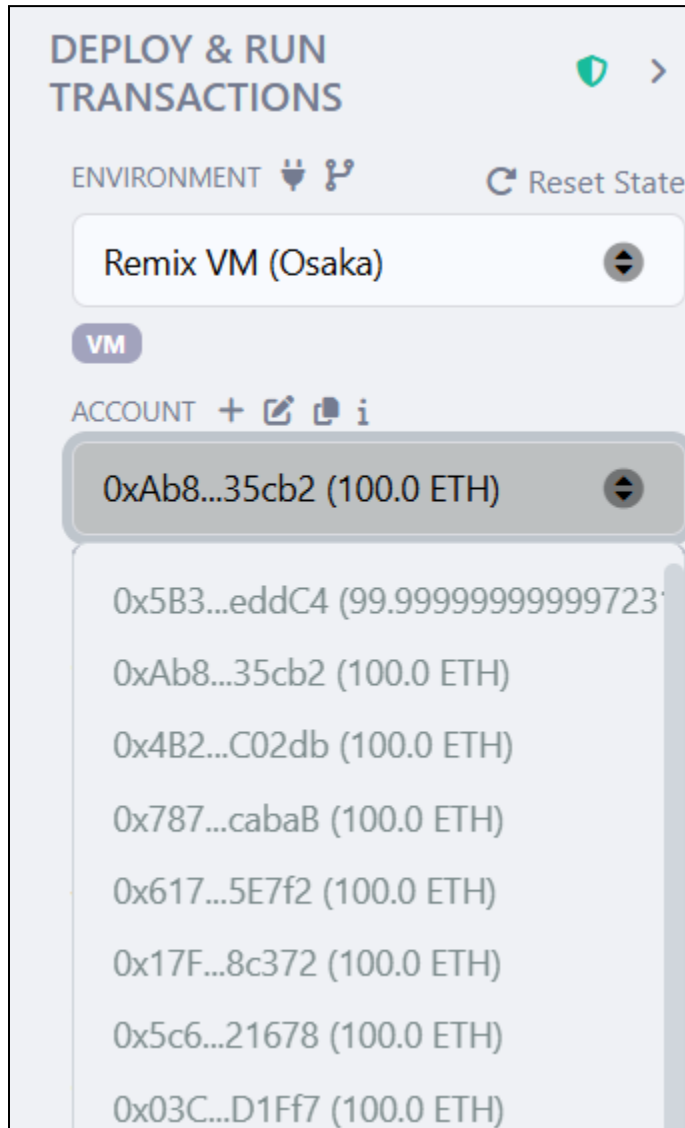
0x03C...D1Ff7 (100.0 ETH)

0x1aE...E454C (100.0 ETH)

0x0A0...C70DC (100.0 ETH)

2. Transfer Ownership

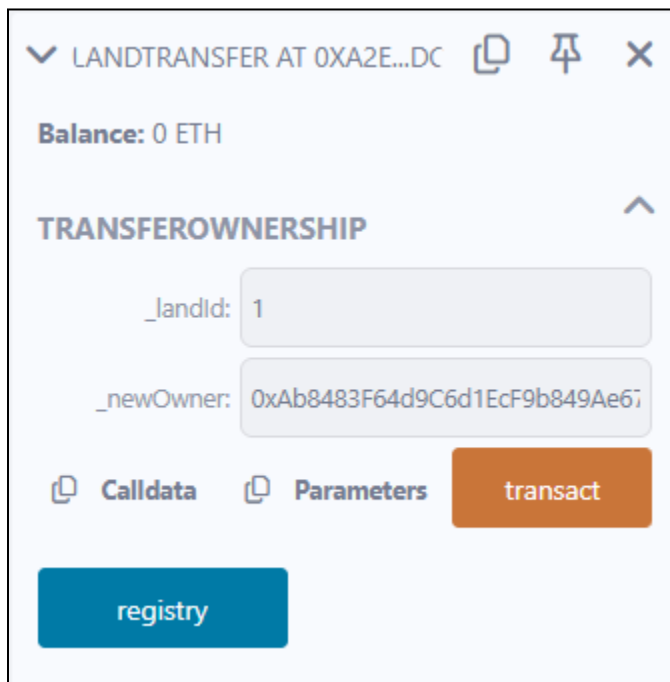
Copy a different wallet address from the account dropdown (this wallet address represents the buyer)



Switch back to the original wallet address(i.e the seller's wallet address) and within the transferOwnership function in the land transfer contract, enter:-

`_landId: 1`

`_newOwner: [The Buyer's Address]`



LANDTRANSFER AT 0XA2E...DC

Balance: 0 ETH

TRANSFEROWNERSHIP

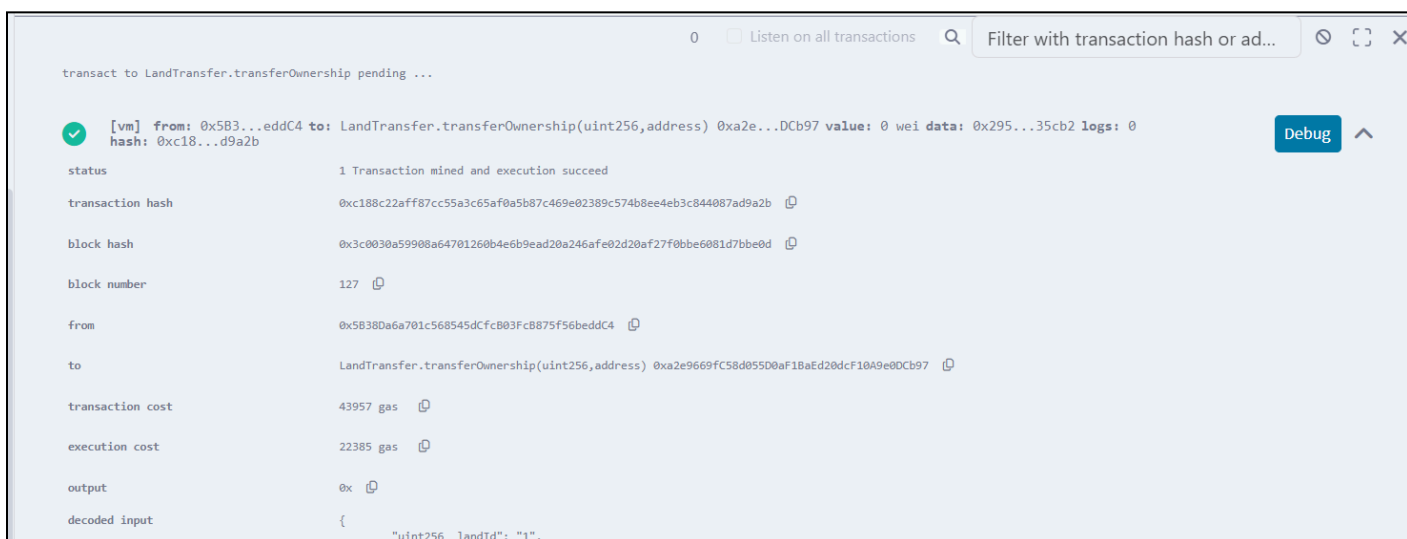
`_landId:` 1

`_newOwner:` 0xAb8483F64d9C6d1EcF9b849Ae67

Calldata Parameters transact

registry

Click on transact to complete the ownership transfer



transact to LandTransfer.transferOwnership pending ...

0 ☐ Listen on all transactions

[vm] from: 0x583...eddC4 to: LandTransfer.transferOwnership(uint256,address) 0xa2e...DCb97 value: 0 wei data: 0x295...35cb2 logs: 0

status 1 Transaction mined and execution succeed

transaction hash 0xc188c22aff87cc55a3c65af0a5b87c469e02389c574b8ee4eb3c844087ad9a2b

block hash 0x3c0030a59908a64701260b4e6b9ead20a246afe02d20af27f0bbe6081d7bbe0d

block number 127

from 0x58380a6a701c568545dCfcB03FcB875f56beddC4

to LandTransfer.transferOwnership(uint256,address) 0xa2e9669fC58d05D0aF1BaEd20dcF10A9e00Cb97

transaction cost 43957 gas

execution cost 22385 gas

output 0x

decoded input { "uint256 landId": "1",

Now, verify the ownership transfer by going to the LandRegistry contract. When we click on the getLand button, we see a different wallet address, indicating that the ownership transfer has taken place and is successfully verified.

The screenshot displays the Remix IDE interface. On the left, the 'REGISTERLAND' panel shows a form with fields for _id (1), _location (Mumbai), _area (500), and _owner (0x5833Da6a701c568545ddfc803fcd). Below the form are buttons for 'updateOwner', 'admin', and 'getLand'. The 'getLand' button is highlighted, and its parameters are shown as 'uint256'. Below the parameters, the output of the function is displayed: '0: uint256: 1', '1: string: Mumbai', '2: uint256: 500', and '3: address: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2'. On the right, the Solidity code editor shows the 'LandTransfer' contract. Below the code, the 'Explain contract' panel displays the transaction details, including the transaction cost (43957 gas), execution cost (22385 gas), output (0x), decoded input ({"uint256 _landId": "1", "address _newOwner": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"}), decoded output ({}), logs ({}), raw logs ({}), and a call to 'LandRegistry.getLand'. The call details show a call from '0x58380a6a701c568545dCfC83FcB875F56beddC4' to 'LandRegistry.getLand(uint256)' with data '0xf02...00001'.

Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.